

LO3 – Functions in Python

Miss Shumaila
FAST NUCES

Concept: What is a Function?

- A function is a reusable block of code that performs a specific task.
- Functions improve modularity and reduce repetition.
- They make large programs easier to understand and maintain.

Basic Syntax

```
def function_name(parameters):  
    statements  
    return value (optional)
```

- The 'def' keyword defines a function.

Example: Simple Function

```
def greet():  
    print('Welcome to Programming')
```

```
greet()
```

Output

- Welcome to Programming

Explanation

1. Function is defined but not executed.
2. When `greet()` is called, the block runs.
3. The print statement executes.

Concept: Parameters

- Parameters allow data to be passed into a function.
- They make functions flexible and reusable.

Example: Addition Function

```
def add(a, b):  
    return a + b
```

```
result = add(5, 3)  
print(result)
```


Output

- 8

Step-by-Step Explanation

1. 5 assigned to 'a', 3 assigned to 'b'.
2. Function computes sum.
3. return sends value back.
4. Value printed.

Business Example: Salary Calculator

```
def calculate_salary(hours, rate):  
    return hours * rate
```

```
print(calculate_salary(40, 500))
```

Output

- 20000

Explanation

- Calculates employee salary based on hours and rate.
- Used in payroll systems.

Business Example: Discount System

```
def apply_discount(price):  
    if price >= 5000:  
        return price - (price * 0.20)  
    return price
```

```
print(apply_discount(6000))
```

Output

- 4800.0

Explanation

- Applies 20% discount if condition satisfied.
- Common in e-commerce systems.

Practice 1 : Square Function

- Write a function that returns the square of a number.

Solution Code

```
def square(num):  
    return num * num  
  
print(square(4))
```

Output

- 16

Step-by-Step Explanation

- 4 multiplied by 4 equals 16.
- Function returns the calculated value.

Practice 3: Count Vowels

- Write a function to count vowels in a string.

Solution Code

```
def count_vowels(text):  
    count = 0  
    for ch in text.lower():  
        if ch in 'aeiou':  
            count += 1  
    return count
```

```
print(count_vowels('University'))
```

Output

- 4

Step-by-Step Explanation

- Loops through each character.
- Checks if character is vowel.
- Increments counter.

Practice 4 : Prime Checker

- Write a function to check if a number is prime.

Solution Code

```
def is_prime(n):  
    if n <= 1: return False  
    for i in range(2, n):  
        if n % i == 0:  
            return False  
    return True
```

```
print(is_prime(7))
```

Output

- True

Step-by-Step Explanation

- Checks divisibility from 2 to $n-1$.
- No divisor found $\rightarrow$ prime.

Practice 5 : Login System

- Create a login function allowing 3 attempts.

Solution Code

```
def login():  
    attempts = 3  
    while attempts > 0:  
        pwd = input('Enter password: ')  
        if pwd == 'admin123':  
            return 'Access Granted'  
        attempts -= 1  
    return 'Account Locked'
```

Output

- Depends on user input

Step-by-Step Explanation

- Uses loop for attempts.
- Checks condition inside function.
- Combines loops + condition + return.

Scope

- Scope defines where a variable can be accessed.
- Two main types:
 - Local Variable
 - Global Variable

Local Variables

- Variables declared inside a function are local.
- They cannot be accessed outside that function.

Local variable Error Example

```
def show():  
    y = 50
```

```
show()  
print(y)
```

- Output:
NameError: name 'y' is not defined

Global Variables

- Variables declared outside all functions are global.
- Accessible throughout program.

Using global Keyword

```
x = 5  
def change():  
    global x  
    x = 10
```

Explanation: global allows modifying global variable.

Output: 10

Example

```
x = 10
```

```
def test():  
    x = 20  
    print("Inside function:", x)
```

```
test()  
print("Outside function:", x)
```

Output:

Inside function: 20

Outside function: 10

Change global variable inside a func

```
x = 10
```

```
def change():  
    global x  
    x = 50
```

```
change()  
print(x)
```

Output: 50

- **Global Variable = Family TV**
Everyone in the house can access it.
- **Local Variable = Bedroom Lamp**
Only available inside that room.

Example

```
count = 5
```

```
def update():  
    count = count + 1  
    print(count)
```

```
update()
```

- Output: Error

Example

```
total = 100
```

```
def add():  
    global total  
    total += 50
```

```
add()  
print(total)  
Output: 150
```

- You are building a salary system.
- `company_bonus = 5000` (Global)
- Each employee has salary inside function (Local)
- Question:
Which variable should be global?
- Answer:
✓ `company_bonus`
Because it applies to all employees.

Common Mistakes

-  Trying to access local variable outside function
-  Modifying global variable without global keyword
-  Confusing same variable names inside and outside functions

Difference between Global and Local variables

Feature	Local Variable	Global Variable
Declared	Inside function	Outside function
Accessible	Only inside function	Entire program
Lifetime	During function execution	Entire program run
Can modify global?	No (without global)	Yes

Difference between Global and Local variables

Variable Type	Where Defined	Keyword Needed
Global variable	Outside function	No (unless modifying)
Local variable	Inside function	No
Modify global variable	Inside function	<code>global</code> required

Types of Argument Passing in Python

- Python supports **four main ways** to pass arguments to functions.
- Python supports:
 - 1 Positional Arguments
 - 2 Keyword Arguments
 - 3 Default Arguments
 - 4 Variable-Length Arguments (*args, **kwargs)

Positional Arguments

- In positional arguments, **values are passed in the same order as parameters** in the function.
Example: `subtract(10, 5)`


```
def student(name, age):  
    print("Name:", name)  
    print("Age:", age)
```

```
student("Ali", 20)
```

```
student(20, "Ali") # Wrong meaning
```

```
Output: Name: 20
```

- Age: Ali
- So **order matters in positional arguments.**

Keyword Arguments

- In keyword arguments, **parameter names are used when passing values.**
- Example: `subtract(a=10, b=5)`

- **Keyword Arguments**

```
def student(name, age):  
    print("Name:", name)  
    print("Age:", age)
```

```
student(age=20, name="Ali")
```

- ✓ Here Python matches arguments using **parameter names**, not position. So order **does not matter**.
- ✓ Improves readability

Mixing Positional and Keyword Arguments

```
def student(name, age, city):  
    print(name, age, city)
```

```
student("Ali", age=20, city="Lahore")
```

- Rule: Positional arguments must come before keyword arguments.
- `student(name="Ali", 20, city="Lahore")` # Error

Default Parameters

- Default arguments allow **parameters to have default values**.
- Default value is used if no argument is passed.

```
def greet(name='Student'):  
    print('Hello', name)  
greet("Ahmed")
```

`greet()`

Output: Hello Student

Hello Ahmed

- **Important Rule of Default Arguments**
- Default parameters must come after non-default parameters.

```
def func(a, b=10):
```

```
    pass
```

```
def func(a=10, b): # Syntax Error
```

```
    pass
```

- Python assigns arguments **from left to right**.
- So when calling a function, Python tries to match arguments to parameters in order.
- **Correct Pattern to Remember**
- Always write parameters in this order:
 - 1. Required parameters
 - 2. Default parameters
 - 3. *args
 - 4. **kwargs
- `def example(a, b, c=5, d=10, *args, **kwargs):`

- **Variable-Length Arguments (*args)**
- Used when number of arguments is unknown.
Python provides:
- *args (Multiple Positional Arguments)
- **kwargs (Multiple Keyword Arguments)
-

*args allows a function to accept **many positional arguments**.


```
def add(*numbers):  
    return sum(numbers)
```

```
print(add(1, 2, 3, 4))
```

The *numbers means the function can accept **multiple positional arguments**.

All the values passed will be stored in **one variable called numbers**.

- ✓ Accepts unlimited positional arguments
- ✓ Stored as tuple

```
def add(*numbers):  
    total = 0  
    for n in numbers:  
        total += n  
    print(total)
```

```
add(2,3,4)
```

Output: 9

- **Variable-Length Keyword Arguments (**kwargs)**
- ****kwargs** allows passing **many keyword arguments**.

```
def student_info(**details):  
    print(details)
```

```
student_info(name="Ali", age=20, city="Lahore")
```

```
Output: {'name': 'Ali', 'age': 20, 'city': 'Lahore'}
```

- ✓ Stored as dictionary
- ✓ Flexible data input

Type	How Arguments Are Passed	Order Important?	Example
Positional	Based on position	Yes	<code>func (10, 20)</code>
Keyword	Using parameter names	No	<code>func (a=10, b=20)</code>
Default	Parameters have default values	Optional	<code>func (name="Ali")</code>
<code>*args</code>	Multiple positional arguments	Yes	<code>func (1, 2, 3)</code>
<code>**kwargs</code>	Multiple keyword arguments	No	<code>func (a=1, b=2)</code>

- Think of a function like **ordering food**.
- Positional argument →
"Burger, Fries" (order matters)
- Keyword argument →
"Fries=1, Burger=1" (order doesn't matter)
- Default argument →
If you don't order a drink → restaurant gives **water by default**
- *args →
You can order **many items**
- **kwargs →
You specify **item names with quantity**

Feature

`*args`

`**kwargs`

Type

Tuple

Dictionary

Accepts

Multiple positional
arguments

Multiple keyword
arguments

Example call

`func (1, 2, 3)`

`func (a=1, b=2)`

```
def example(a, b=10, *args, **kwargs):  
    print(a, b)  
    print(args)  
    print(kwargs)
```

```
example(1,2,3,4,name="Ali")
```

Output: 1 2

(3, 4)

{'name': 'Ali'}

Argument Passing with Mutable vs Immutable Types

Immutable (int, str, tuple)

Example:

```
def change(x):  
    x = 100
```

```
num = 10  
change(num)  
print(num)
```

- **Output: 10**

When immutable data types are passed to a function:

- The function receives **a copy of the value**
- Changes inside the function **do not affect the original variable**

Immutable types

Examples of immutable types:

- Float
- int
- str
- Tuple

Immutable means:

- The value cannot be changed directly.
- Instead, Python creates a **new value** rather than modifying the old one.

Mutable (list, dictionary)

```
def add_item(lst):  
    lst.append(100)
```

```
numbers = [1, 2, 3]  
add_item(numbers)  
print(numbers)  
Output: [1, 2, 3, 100]
```

Example

```
def change(lst):  
    lst[0] = 100
```

```
numbers = [10, 20, 30]  
change(numbers)  
print(numbers)
```

Output: [100, 20, 30]

Mutable Data Types

- Lists are **mutable**.

Mutable means:

The object can be changed after it is created.

Examples of mutable types:

- list
- dictionary
- set

Comparison

Type	Mutable?	Change inside function affects original?
int	No	No
string	No	No
tuple	No	No
list	Yes	Yes
dictionary	Yes	Yes
set	Yes	Yes

Multiple Return Values

```
def values():  
    return 1, 2, 3
```

```
a, b, c = values()  
print(a, b, c)
```

Output: 1 2 3

Mapping

Parameter

Defined in function

Placeholder variable

Example: x

Argument

Passed when calling function

Actual value

Example: 10

Global variable

x = 50

```
def student_info(name, age=20, *courses, **details):
```

```
    # Local variable
```

```
        y = 10
```

```
    print("Inside function:")
```

```
    # Accessing global variable
```

```
    print("Global variable x:", x)
```

```
    # Accessing local variable
```

```
    print("Local variable y:", y)
```

```
    # Positional argument
```

```
    print("Name:", name)
```

```
    # Default argument (if not provided)
```

```
    print("Age:", age)
```

```
    # Variable-length positional arguments (*args)
```

```
    print("Courses:", courses)
```

```
    # Variable-length keyword arguments (**kwargs)
```

```
    print("Additional Details:", details)
```

Function call

```
student_info("Ali", 22, "Math", "Physics", city="Lahore", grade="A")
```


Output:

Inside function:

Global variable x: 50

Local variable y: 10

Name: Ali

Age: 22

Courses: ('Math', 'Physics')

Additional Details: {'city': 'Lahore', 'grade': 'A'}

Examples:

```
def add_item(item, lst=[]):  
    lst.append(item)  
    return lst
```

```
print(add_item(1))  
print(add_item(2))  
print(add_item(3))
```

Output:

```
[1]
```

```
[1, 2]
```

```
[1, 2, 3]
```

Example

```
def add_item(item, lst=None):  
    if lst is None:  
        lst = []  
        lst.append(item)  
    return lst
```

```
print(add_item(1))  
print(add_item(2))  
print(add_item(3))
```

Output:

```
[1]  
[2]  
[3]
```

Version

```
lst=[]  
lst=None
```

Output

```
[1], [1, 2], [1, 2, 3]  
[1], [2], [3]
```

Example

```
x = 10
```

```
def test(a, b=5, *args, **kwargs):  
    x = a + b  
    print("A:", x)  
    print("B:", args)  
    print("C:", kwargs)
```

```
test(3, 7, 9, 11, name="Sara")  
print("D:", x)
```

Output:

A: 10

B: (9, 11)

C: {'name': 'Sara'}

D: 10

Concept

Example

Global variable

```
x = 10
```

Local variable

```
x = a + b
```

Positional arguments

```
3, 7
```

Default argument

```
b=5
```

*args

```
(9, 11)
```

**kwargs

```
{ 'name' : 'Sara' }
```

Scope

local `x` does not affect global `x`